

PLATFORM DEVELOPMENT ROADMAP

The path from working demo to production platform.

A phased execution plan covering the five confirmed workstreams still open on Veridex, plus the additional hardening, observability, and release-engineering tracks required to call this platform enterprise-ready.

9	14-18	4	100%
---	-------	---	------

DELIVERY PHASESWEEKS, FULL SCOPENEWLY IDENTIFIED GAPSTRACED TO STATUS REPORT

01 — EXECUTIVE SUMMARY

Where the platform actually stands.

The core orchestration engine works. The DAG renders in real time, the frontend talks to the backend over SSE, and the local data layer is seeded and functional. That's a real foundation — most of what remains is not "does it work," it's "does it hold up in production, under real data and real load." This document takes the five open items from the status report and restructures them into a build order that respects dependencies, then adds four tracks — security, observability, testing, and release engineering — that every one of the original five quietly depends on.

5

Confirmed gaps from status report

4

Additional gaps surfaced by this plan

3

Items blocking enterprise sale today

The one-sentence version: data pipeline and governance are the two items that make this a product rather than a demo — everything else (UI polish, dashboard wiring, cloud migration) is necessary but not differentiated. Sequence accordingly: harden the data and permission model first, then let the interface catch up to what the backend can actually do.

WHAT "DONE" ALREADY COVERS

- Real-time orchestration DAG rendering, confirmed working end-to-end in the browser, with the frontend connected to the backend via SSE and CORS correctly configured.
- Local postgres:16-alpine container, seeded with representative Organization / Workspace / Project data, plus an RBAC schema modeled in the database layer.
- Backend scaffolding present for Connectors (GitHub, Slack, Jira), Qdrant, and Neo4j.
- The timeline/goal orchestrator component functional in the dashboard.

Read this document as a build order, not a checklist: the next page breaks all nine workstreams into the five originally confirmed and the four this plan surfaces, each with the dependency that governs when it can realistically start.

01 — EXECUTIVE SUMMARY, CONTINUED

Nine workstreams, two lists.

Five of these were already on the status report. Four were not. Laid out together, the dependency column is the one that actually determines sequencing — it's why Phase 2 (governance) can't wait for Phase 3 (cloud migration) to finish, and why security work starts alongside both rather than after them.

• THE FIVE CONFIRMED GAPS, REFRAMED AS AN EXECUTION ORDER

#	WORKSTREAM	PRIORITY	DEPENDS ON
1	Knowledge Ingestion & Vector Pipeline (RAG)	CRITICAL	— (foundation)
2	Enterprise Governance & Policy Enforcement (OPA)	CRITICAL	RBAC schema
3	Production Cloud Database Migration	HIGH	Phase 2 policies
4	Frontend Dashboard Integration	HIGH	Phases 1–3
5	Advanced UI/UX Implementation (PDF Designs)	MEDIUM	Phase 4

• FOUR GAPS THIS PLAN ADDS **NOT IN ORIGINAL REPORT**

#	WORKSTREAM	WHY IT'S NOT OPTIONAL
6	Security Hardening & Secrets Management	Cloud migration + multi-tenancy without this is a breach waiting to happen.
7	Observability & Agent Telemetry	Can't debug a multi-agent DAG in production without traces and metrics.
8	Testing, QA & Load Validation	RAG and policy engines fail silently and expensively without test coverage.
9	CI/CD & Release Engineering	Manual deploys don't survive contact with an enterprise SLA.

Worth sitting with: none of the four added workstreams appear anywhere in the original status report, and all four are the kind of gap that surfaces during a security review or a production incident rather than a demo — by which point they're far more expensive to fix.

02 — SEQUENCING

Nine phases, eighteen weeks.

Phases 1 and 2 can start in parallel — they touch different parts of the stack and both block everything downstream. Phase 6 (security) runs alongside 2-3 rather than after them, since secrets management has to exist before anything touches a real cloud database. Observability (7) and testing (8) are threaded through every phase rather than bolted on at the end — the timeline below shows where each starts, not a separate strip, because in practice they're continuous.



Reading this chart: bars starting mid-track (Phases 3, 4, 5, 9) are gated on the phase before them clearing its Definition of Done, not on a calendar date — if Phase 1 slips, everything to its right slips with it. Phases 7 and 8 are drawn wide because they're not phases in the traditional sense; they're standing practices that start once there's something to observe or test, and never really end.

PHASE 1 · WEEKS 1-4 · CRITICAL PATH

Wire the RAG pipeline end to end. 01

The connector scaffolding and vector/graph stores exist but aren't talking to each other. This phase makes an agent's context fetch actually pull real data — split, embedded, and stored — rather than hitting an empty index.

DURATION
4 weeks

OWNER
Backend / ML

BLOCKS
Phases 3, 4

RISK
High

CORE WORKSTREAMS

- ☐ **Text splitting & normalization layer.**
Recursive/semantic chunking per source type (code, Slack threads, Jira tickets, wiki docs) with overlap tuned per content shape, not one splitter for everything.
- ☐ **Embedding generation via Gemini models.**
Batched embedding calls with retry/backoff, request-level caching to avoid re-embedding unchanged content, and a fallback path if the embedding API rate-limits mid-ingest.
- ☐ **Qdrant write path.**
Collection-per-workspace or filtered single collection (decide based on tenant count), payload schema for source/permissions metadata, upsert-not-duplicate logic on re-ingestion.
- ☐ **Neo4j graph write path.**
Entity/relationship extraction from ingested content (people, repos, tickets, channels) so agents can traverse relationships, not just similarity-match chunks.
- ☐ **Agent-facing retrieval API.**
The actual function an agent calls at inference time — hybrid search (vector + graph), reranking, and a token-budget-aware context assembler.
- ☐ **Ingestion job orchestration.**
Queue-backed workers (not synchronous HTTP handlers) so a large repo or Slack export doesn't block the API process; per-connector rate limiting to respect GitHub/Slack/Jira API quotas.

STACK DECISIONS

Vector store	Qdrant
Graph store	Neo4j
Embeddings	Gemini
Queue	Redis / Celery
Chunking	Per-source strategy

DELIVERABLES

- Ingestion service with connector adapters live
- Retrieval API consumed by ≥1 real agent
- Eval harness measuring retrieval precision

Definition of Done:

a connected GitHub repo, Slack workspace, and Jira project can each be ingested, and a live agent query returns cited, relevant context from all three within a measured latency budget.

RISKS TO PLAN AROUND

- Silent quality failure:** a broken chunking strategy doesn't throw errors — it just returns irrelevant context and the agent hallucinates confidently. Build a retrieval-quality eval set (Phase 8) before calling this "done."
- Re-ingestion cost:** without content hashing, every connector sync re-embeds everything. At scale this is an unbounded Gemini API bill.

PHASE 2 · WEEKS 1-3.5 · CRITICAL PATH

Make permissions actually enforce. 02

The RBAC schema describes who should be able to do what. It doesn't yet stop anyone from doing anything. This phase builds the OPA-style policy evaluation layer that turns the schema into real, hard multi-tenant enforcement at the middleware level.

DURATION
3.5 weeks

OWNER
Backend / Platform

BLOCKS
Phase 3, sale-readiness

RISK
Critical

• CORE WORKSTREAMS

☐ **YAML policy schema design.**

Define the policy document format up front — subject, resource, action, condition — so policies are versionable and reviewable in PRs, not buried in code.

☐ **Policy evaluation engine.**

Either embed Open Policy Agent directly (via its Go binary or WASM build) or implement a compatible Rego-subset evaluator in Python — decide based on ops complexity budget.

☐ **FastAPI middleware integration.**

A dependency-injected guard that resolves subject + resource + action on every request before the route handler runs, with policy decisions cached per-request to avoid re-evaluation overhead.

☐ **Hard tenant isolation.**

Row-level security or query-scoping at the ORM layer so a policy bug fails closed (denies access) rather than fails open (leaks cross-tenant data) — this is the actual "hard" in hard multi-tenancy.

☐ **Policy authoring & audit UI.**

A minimal admin surface for org admins to view and adjust their own policies, plus an audit log of every allow/deny decision for compliance review.

STACK DECISIONS

Policy format	YAML → Rego-compatible
Engine	OPA (embedded) or subset
Enforcement point	FastAPI middleware
Isolation model	Default-deny

DELIVERABLES

- Policy engine wired into every protected route
- Cross-tenant access attempt test suite, all passing
- Audit log of allow/deny decisions

Definition of Done: a scripted adversarial test — user from Org A attempting to read Org B's projects, agents, and vector data — is denied at every layer, with the denial logged.

• RISKS TO PLAN AROUND

Fail-open is the enterprise dealbreaker. Any path where a missing or malformed policy defaults to "allow" is a cross-tenant data leak waiting for a customer's security review to find. Every evaluation path needs an explicit default-deny.

Performance tax: policy evaluation on every request adds latency. Benchmark early — this needs to add single-digit milliseconds, not tens.

PHASE 3 · WEEKS 4-6

03

Move off the laptop, safely.

The local `postgres:16-alpine` container has done its job — it proved the schema and let development move fast. Now it needs to become a managed service with backups, connection pooling, and a migration path that doesn't lose the seeded data model along the way.

DURATION

2 weeks

OWNER

Platform / DevOps

DEPENDS ON

Phase 2 policies

RISK

Medium

• CORE WORKSTREAMS

☐ Provider selection & provisioning.

Score Supabase, AWS RDS, and Cloud SQL against actual requirements — connection pooling needs, read-replica story, existing cloud footprint — not just feature checklists. Document the decision.

☐ Schema & migration tooling.

Formalize migrations (Alembic or equivalent) if not already in place, so the cloud instance is reached through versioned migrations, not a manual dump-and-restore.

☐ Connection pooling & environment config.

PgBouncer or provider-native pooling; environment-based connection strings so local dev, staging, and production never accidentally point at the same instance.

☐ Backup, PITR & disaster recovery.

Automated backups with point-in-time recovery enabled from day one, plus one documented and tested restore drill — not just a checkbox in the provider console.

☐ Cutover plan.

Dual-write or read-from-new / write-to-both window during transition, with a clear rollback trigger if the managed instance shows unexpected latency under real traffic.

PROVIDER SHORTLIST

Supabase	Fast, Postgres-native
AWS RDS	Deep AWS integration
Cloud SQL	If GCP-hosted already

DELIVERABLES

- Managed Postgres instance, versioned migrations applied
- Automated backup + one tested restore drill
- Documented cutover runbook

Definition of Done: the application runs entirely against the managed instance, a restore-from-backup drill has been performed successfully, and the local Docker Postgres is demoted to dev-only.

• RISKS TO PLAN AROUND

Dummy data ≠ production data shape. Seeded Organizations/Workspaces/Projects data is small and clean. Load-test the migration and the queries against realistic volume before trusting the cutover.

Connection exhaustion. Serverless/managed Postgres tiers often cap connections lower than expected — validate pooling handles the orchestrator's concurrent agent load.

PHASE 4 · WEEKS 6.5-10.5

Give the rest of the app a backend.

The orchestrator view is real and working. Everything around it — auth, project lists, agent logs, telemetry — is still waiting to be connected. This phase wires the remaining dashboard surface to real data, now that Phases 1-3 mean there's real data and real permissions behind it.

DURATION

4 weeks

OWNER

Frontend

DEPENDS ON

Phases 1-3

RISK

Medium

CORE WORKSTREAMS

☐ Real authentication flow.

Session/JWT handling wired to the Phase 2 policy layer — login, org switching, and token refresh, replacing whatever stub auth exists today.

☐ Project & workspace listing.

Live queries against the (now cloud-hosted) database, scoped through the policy engine so users only ever see what they're permitted to see — no client-side filtering as the security boundary.

☐ Agent log viewer.

Streamed or paginated view into agent execution logs, tied to the observability work in Phase 7 rather than built as a one-off log tailer.

☐ Telemetry & metrics display.

Token usage, latency, and cost-per-run surfaced per project/workspace — this is what lets a customer trust the platform with production workloads.

☐ Empty, loading, and error states.

Every list and panel needs a designed empty state and error state, not just a happy-path render — these are usually 30% of the actual engineering effort on integration work.

RISKS TO PLAN AROUND

Auth as an afterthought. If auth is bolted onto existing components rather than designed in from the route layer, expect a second pass once Phase 2's policy engine is fully live.

SURFACES IN SCOPE

Auth / login	New
Project list	New
Agent logs	New
Telemetry panel	New
Orchestrator DAG	Already live

DELIVERABLES

- Full auth flow, session-aware routing
- Project/workspace lists backed by live queries
- Agent log + telemetry views, connected to Phase 7

Definition of Done: a new user can register, land in the correct org context, see only their permitted projects, and inspect a real agent run's logs and cost — with zero mock data remaining in the dashboard.

PHASE 5 · WEEKS 10.5-13

Make it feel like the reference design.

This is deliberately sequenced last among the frontend work. Polishing animations and layout on components that are about to be rewired to real data (Phase 4) risks throwaway work. Once the dashboard is functionally complete, this phase closes the gap to the PDF reference design.

DURATION
2.5 weeks

OWNER
Frontend / Design

DEPENDS ON
Phase 4

RISK
Low

● CORE WORKSTREAMS

☐ Motion & micro-interaction pass.

The "amusing" animations from the reference — DAG node transitions, state-change feedback, loading choreography — implemented with a shared animation primitives library rather than one-off CSS per component.

☐ Layout & spacing fidelity.

Complex layout elements from the PDF (grid structures, panel arrangements, responsive breakpoints) matched against the actual reference file, page by page.

☐ Design token consolidation.

Pull colors, spacing, and typography from the PDF into a single design-tokens file so this polish doesn't drift from the reference the next time a component is touched.

☐ Performance budget for animation.

GPU-accelerated transforms only, animation-heavy views tested on mid-tier hardware — "amusing" shouldn't mean "janky on anything but a dev machine."

● WHY THIS IS SEQUENCED LAST, NOT FIRST

Polish is durable once it's on top of real functionality.

Building this against the current dashboard would mean redoing it once Phase 4's real data, empty states, and error states land. Sequencing it after means every animation and layout decision is made against the actual final component tree.

REFERENCE SOURCE

Format PDF UI spec

Fidelity target Pixel + motion

DELIVERABLES

- Design tokens extracted & applied platform-wide
- Animation pass on DAG, dashboard, and transitions
- Side-by-side fidelity review vs. PDF reference

Definition of Done: a side-by-side comparison of each key screen against its PDF reference page shows matching layout, spacing, and motion, signed off by whoever owns the original design.

03 — BEYOND THE STATUS REPORT

The gaps nobody wrote down.

Every one of the five confirmed items quietly assumes something else exists: a way to store secrets safely once there's a real cloud database, a way to see what an agent actually did once logs stop being a local terminal, a way to trust the RAG pipeline before customer data flows through it, and a way to ship changes without someone manually running commands against production. These four phases aren't scope creep — they're the load-bearing walls the other five are quietly standing on.

06

Security Hardening & Secrets Management

07

Observability & Agent Telemetry

08

Testing, QA & Load Validation

09

CI/CD & Release Engineering

Why these matter more than they look: a security review, an SLA negotiation, or a single production incident will surface all four of these in the first week of an enterprise pilot — the only choice is whether Veridex is ready for that conversation, or discovering the gap live in front of a customer.

• HOW THESE INTERLEAVE WITH PHASES 1-5

- **Security (6)** starts in week 1.5 — before the cloud DB migration (Phase 3) touches anything, secrets and credential handling need a real home.
- **Observability (7)** starts as soon as there's an ingestion job or policy decision worth tracing — effectively alongside Phase 1.
- **Testing (8)** ramps up once Phase 1's retrieval pipeline and Phase 2's policy engine have something to test against — around week 8.
- **CI/CD (9)** is the final phase — it assumes there's a stable enough surface (post Phase 5) to build a real pipeline around.

PHASE 6 · WEEKS 1.5-4 · RUNS ALONGSIDE PHASES 2-3

Assume you'll be breached. Plan accordingly.

06

Neither the original five items nor most roadmaps mention this explicitly, but a cloud database migration and hard multi-tenancy without a real secrets and credential story is how connector API keys end up in a git history. This phase closes that gap before Phase 3 gives it a real target.

DURATION
2.5 weeks

OWNER
Platform / Security

BLOCKS
Phase 3 cutover

RISK
Critical

• CORE WORKSTREAMS

☐ Secrets manager adoption.

Move connector tokens (GitHub, Slack, Jira), database credentials, and the Gemini API key out of environment files and into a real secrets manager (AWS Secrets Manager, GCP Secret Manager, or Vault) with rotation policy.

☐ Connector token encryption at rest.

Per-tenant connector credentials encrypted in the database, not stored as plaintext columns — this is the part of "hard multi-tenancy" that policy enforcement alone doesn't cover.

☐ Dependency & container scanning.

Automated vulnerability scanning on both Python/JS dependencies and Docker images, wired into CI once Phase 9 exists — flagged manually before then.

☐ Input validation on agent-facing endpoints.

Prompt-injection-aware validation on any endpoint where external content (Slack messages, Jira tickets, GitHub issues) flows into an agent's context — a direct risk given Phase 1's ingestion pipeline.

☐ TLS & network policy review.

Enforce TLS everywhere between services, review that the managed database (Phase 3) isn't reachable from the public internet by default.

PRIORITY ORDER

- | | |
|--------------------------|----------|
| 1. Secrets manager | Week 1.5 |
| 2. Token encryption | Week 2 |
| 3. Input validation | Week 3 |
| 4. Scanning + TLS review | Week 4 |

DELIVERABLES

- All secrets in a managed secrets store, zero in plaintext
- Encrypted connector credentials, per tenant
- Documented input-validation policy for agent context

Definition of Done: a credential audit finds zero plaintext secrets in code, config, or the database, and a scripted prompt-injection attempt via an ingested Slack message is safely neutralized.

• RISKS TO PLAN AROUND

Connector tokens are the highest-value target in this system. A leaked GitHub or Slack token from one tenant's connector is a breach of that tenant's entire external data footprint, not just Veridex data.

PHASE 7 • WEEKS 1.5-12.5 • CONTINUOUS PRACTICE

See what the agents are actually doing.

07

Phase 4's telemetry panel needs something real to display. This phase is the instrumentation work underneath it — distributed tracing across the orchestration DAG, structured logging, and metrics that make a multi-agent system debuggable rather than a black box.

DURATION
Continuous**OWNER**
Platform**FEEDS**
Phase 4 dashboard**RISK**
Medium

CORE WORKSTREAMS

☐ Distributed tracing across the DAG.

OpenTelemetry instrumentation so a single orchestration run — spanning multiple agents, a retrieval call, and a policy check — is traceable as one connected span tree, not scattered log lines.

☐ Structured logging standard.

One consistent log schema (JSON, with tenant/run/agent IDs on every line) across every service, so the Phase 4 log viewer can actually filter and correlate rather than grep raw text.

☐ Cost & token metrics.

Per-run and per-tenant tracking of Gemini token usage and embedding costs — this is the specific metric that turns into a billing model and a customer-facing cost dashboard.

☐ Alerting on the failure modes that matter.

Alerts for policy-engine fail-open events, ingestion pipeline stalls, and agent run timeouts — the failure modes specific to this system, not generic CPU/memory thresholds alone.

☐ Retrieval quality monitoring.

Ongoing tracking of the Phase 1 eval metrics in production, not just at build time — retrieval quality can degrade silently as real data volume grows.

SUGGESTED STACK

Tracing	OpenTelemetry
Metrics	Prometheus + Grafana
Logs	Structured JSON, central sink
Alerting	Grafana / PagerDuty

DELIVERABLES

- Full request/agent trace across one orchestration run
- Per-tenant cost & token dashboards
- Alerting on the five system-specific failure modes

Definition of Done: any single orchestration run can be traced end to end from trigger to final agent output, with cost, latency, and retrieval quality all visible in one place.

WHY THIS CAN'T WAIT UNTIL LATER

Debugging a multi-agent system after the fact is close to impossible without tracing built in from the start — by the time Phase 4 needs data to display, the instrumentation needs to already be capturing it, not being retrofitted.

PHASE 8 · WEEKS 8-16 · CONTINUOUS PRACTICE

Trust, but verify — especially the RAC pipeline.

RAG pipelines and policy engines are exactly the kind of system that looks fine in a demo and fails expensively in production — because failures are silent (bad retrieval) or catastrophic (a policy fail-open) rather than a visible crash. This phase builds the test discipline neither category of failure survives.

DURATION

Continuous, ramps week 8

OWNER

All engineering

VALIDATES

Phases 1, 2, 3

RISK

High if skipped

CORE WORKSTREAMS

☐ **Policy engine adversarial test suite.**

Automated, scripted cross-tenant access attempts run on every deploy — the single highest-value test suite in this entire roadmap, given what a fail-open means (Phase 2).

☐ **Retrieval quality regression tests.**

A golden set of query → expected-context pairs per connector type, run on every change to the chunking or embedding pipeline, so a "silent" quality regression actually gets caught.

☐ **Load testing the orchestration DAG.**

Concurrent multi-agent runs against the Phase 3 cloud database and Phase 2 policy engine — validate connection pooling and policy-check latency hold up under realistic concurrent tenant load.

☐ **Integration tests across connector boundaries.**

GitHub/Slack/Jira connector adapters tested against recorded API fixtures, so connector-side rate limits or schema changes are caught in CI, not in a customer's ingestion run.

☐ **Chaos / failure-injection testing.**

Deliberately kill the vector store, the graph store, or a connector mid-run to confirm the orchestrator degrades gracefully rather than corrupting state.

WHERE THE HIGHEST LEVERAGE IS

If only one test suite gets built this quarter, it's the policy adversarial suite. Every other gap in this document is expensive to fix; a multi-tenant data leak is the one that's also reputationally unrecoverable.

TEST PYRAMID, THIS SYSTEM

Adversarial (policy)	Highest priority
Retrieval quality	High priority
Load / concurrency	Pre-launch gate
Unit / integration	Continuous

DELIVERABLES

- Cross-tenant adversarial suite, run on every deploy
- Retrieval quality golden set with pass/fail threshold
- One documented load test at target concurrent-tenant scale

Definition of Done: the adversarial policy suite and retrieval quality suite both run automatically in CI (Phase 9) and block merges on failure; one load test has been run at the target enterprise-pilot concurrency.

PHASE 9 · WEEKS 15-18 · FINAL PHASE

Ship without anyone SSHing into anything.

09

Everything before this phase assumed a person could run a migration, deploy a service, or execute a test suite by hand. That doesn't survive an actual enterprise SLA. This phase is what turns the previous eight phases' work into something that deploys itself, reliably, with a rollback plan.

DURATION
3 weeks

OWNER
Platform / DevOps

DEPENDS ON
Phases 1-8 stable

RISK
Low

• CORE WORKSTREAMS

☐ CI pipeline: build, test, scan.

Every merge runs the Phase 8 test suites (including the adversarial policy suite) and the Phase 6 dependency scan, blocking on failure — the previous two phases only have teeth once this exists.

☐ Automated migration deployment.

Database migrations (Phase 3's Alembic setup) applied automatically as part of deploy, with an automatic pre-migration backup and a tested rollback path.

☐ Containerized, reproducible deploys.

Every service (API, ingestion workers, frontend) built as a versioned container image, deployed the same way in staging and production — no more environment drift between a developer's machine and prod.

☐ Staging environment mirroring production.

A real staging tier with its own cloud database and policy configuration, so the first time a change touches production isn't also the first time it's been tested against real infrastructure.

☐ Rollback & incident runbook.

A documented, tested procedure for rolling back a bad deploy — including the database migration — so a Friday-afternoon incident has a known playbook rather than an improvised one.

PIPELINE STAGES

- | | |
|---------------------------------|------------------------|
| 1. Build + scan | Every PR |
| 2. Test (Phase 8 suites) | Every PR |
| 3. Deploy → staging | On merge |
| 4. Deploy → production | Gated / manual approve |

DELIVERABLES

- Full CI pipeline gating on tests + security scan
- Automated, reversible migration deployment
- Staging environment + tested rollback runbook

Definition of Done: a code change can go from merge to production, through staging, fully automated, with a rollback executed successfully at least once in a drill.

• WHY THIS IS GENUINELY THE LAST PHASE

A release pipeline automates whatever process already exists. Building this before Phases 1-8 stabilize means automating a moving target. Once the platform's surface is stable, this phase is what makes every future change fast and safe instead of risky.

04 — OWNERSHIP AT A GLANCE

Who's accountable for what.

A rough allocation across functions, useful for staffing conversations. Most phases need more than one function — this shows primary ownership, not exclusive involvement.

#	PHASE	PRIMARY OWNER	DURATION	PRIORITY
1	Vector & RAG Pipeline	Backend / ML	4 weeks	CRITICAL
2	Governance & OPA Enforcement	Backend / Platform	3.5 weeks	CRITICAL
3	Cloud Database Migration	Platform / DevOps	2 weeks	HIGH
4	Dashboard Integration	Frontend	4 weeks	HIGH
5	UI/UX Polish (PDF spec)	Frontend / Design	2.5 weeks	MEDIUM
6	Security Hardening ADDED	Platform / Security	2.5 weeks	CRITICAL
7	Observability & Telemetry ADDED	Platform	Continuous	MEDIUM
8	Testing, QA & Load ADDED	All Engineering	Continuous	HIGH
9	CI/CD & Release Engineering ADDED	Platform / DevOps	3 weeks	MEDIUM

THE THREE ITEMS THAT BLOCK AN ENTERPRISE SALE, SPECIFICALLY

1. Hard multi-tenancy (Phase 2). Any enterprise security questionnaire asks this directly. An RBAC schema without enforcement is not an answer a security team accepts.

2. Secrets management (Phase 6). Equally a standard security-review question, and one that's cheap to fix now versus expensive to fix after a customer asks.

3. Backup & disaster recovery (Phase 3). "What's your RPO/RTO" is a question every enterprise procurement process asks, and it requires the managed database plus a tested restore, not just a migration.

Nine phases. One honest sequence.

Nothing here changes what needs to get built — the five items from the status report are still the five items. What this document adds is an order that respects what actually depends on what, and a name for the four things every one of those five was quietly assuming would already exist. Build the data and permission layer first; let the interface catch up to what the backend can prove it can do.

18

Weeks to close every phase, original five plus four added, run in parallel where dependencies allow.

2

Phases that can start today, in parallel: the RAG pipeline and the governance engine.

1

Test suite worth building before any other, if time is short: the cross-tenant adversarial suite.